Rust OS Book

Philipp Oppermann 氏の公開チュートリアル `Writing an OS in Rust` と、その周辺の `rust-osdev` エコシステムをもとに、`Boot` から `Paging` と `Heap` までを 1 冊に再構成した学習用ノートです。`compilerbook` の次に読むことを想定し、言語実装の次に「そのコードが動く場」を自分で作る流れになるように章立てしています。

> **編集注記**
> この文書は公式記事の逐語訳ではありません。公開されている章順と設計意図を尊重しつつ、単一の Typst PDF として読みやすいように再構成した版です。記事ごとの分割をそのまま写すのではなく、教材として連続して読めるように説明順序と粒度を調整しています。

目次

Contents

はじめに 3
- この教材の主題 3
- 何を作るのか 4
- なぜ Rust なのか 4
- カーネルはどこにいるのか 4
- 本書の範囲 4
- どのように読むか 5
- `compilerbook` とのつながり 5
- 参考にしている公開 OSS 6
- 各章の見取り図 6

フリースタンディング Rust バイナリとブート 6
- 普通の Rust バイナリが暗黙に依存しているもの 6
- `no_main` とエントリポイント 7
- パニック時に何をするか 7
- ブートまでに必要な層 7
- BIOS/UEFI をどこまで知るべきか 8
- 開発環境の考え方 8
- 最小カーネルの形 8
- `hlt` ループの意味 9
- ターゲット指定とリンク 9
- 最初に画面へ何か出す意味 9
- 観察可能性を最優先する 9
- この段階での確認項目 10
- 次章への橋渡し 10

文字出力とテスト基盤 10
- VGA テキストモードはなぜ教材向きか 10
- `volatile` 書き込みの意味 10
- Writer 抽象化 11

書式付き出力をどう実現するか 11
グローバル出力と排他制御 11
シリアル出力も早めに持つ 11
カーネルでのテストはなぜ難しいか 12
テストランナの最小像 12
パニックとテスト失敗 12
最初に書くべきテスト 12
`unsafe` をどこへ閉じ込めるか 13
画面出力で気を付ける落とし穴 13
テストを先に整える価値 13
この段階での到達基準 13
例外、ダブルフォールト、ハードウェア割り込み 14
例外と割り込みの違い 14
IDT の役割 14
ブレークポイント例外から始める理由 14
ダブルフォールトは何が怖いのか 14
IST と専用スタック 15
例外ハンドラで何を出すか 15
ハードウェア割り込みの入口 15
タイマ割り込み 15
キーボード割り込み 16
End of Interrupt を忘れない 16
割り込みコンテキストでのロック 16
ページフォールトは後半の主役 16
この章で得られる感覚 17
次章への橋渡し 17
ページングとヒープ確保 17
なぜページングが必要なのか 17
x86_64 の 4 段ページテーブル 17
ブートローダが渡すもの 18
物理メモリをどう見えるようにするか 18
`Mapper` の見方 18
仮想アドレス変換を自分で辿る 18
新しい写像を作る 18
フレームアロケータ 19
ページングと安全性 19
ヒープはなぜすぐ使えないのか 19
`alloc` を使えるようにする 19
どんなテストを書くべきか 20
ページフォールトをどう読むか 20
ここまでで得られるもの 20
次に進むなら 20
付録と次の一歩 21
推奨するコード構成 21
開発ループの基本形 21
デバッグの進め方 21

QEMU とログの使い分け 22
よくあるつまずき 22
演習課題 22
章ごとの復習ポイント 23
用語小事典 23
ここから先に伸ばす方向 23
比較しながら学ぶとよい OSS 24
`compilerbook` との往復 24
FAQ 24
Rust で OS を書くのは遠回りではないか 24
最初からユーザ空間やファイルシステムまで進むべきか 24
実機で動かすべきか 24
どうしてもダブルフォールトで落ちるときは 24
参考資料 24
おわりに 25

はじめに

この文書は、Rust で小さなカーネルを作りながら OS の骨格を理解するためのノートです。目標は、巨大な機能を備えた実用 OS をいきなり完成させることではありません。起動、画面出力、例外、割り込み、ページング、ヒープ確保までを自力で通し、「ユーザプログラムを動かす以前に、土台では何が起きているのか」を自分の言葉で説明できるようになることを目指します。

`compilerbook` で言語を作ったあとに OS を学ぶ流れには相性のよさがあります。コンパイラでは、ソースコードを別の表現へ変換し、最終的に機械が理解できる命令列を出力しました。OS では、その命令列がどうやって起動し、どのメモリ空間で動き、異常や外部入力にどう反応するかを作っていきます。前者が「プログラムを作る側」の視点なら、後者は「プログラムが走る場を作る側」の視点です。

この教材の主題

本書の主題は、Rust の型システムと `unsafe` を使い分けながら、x86_64 向けの小さなカーネルを段階的に育てることです。表面上は VGA へ文字を出したり、IDT を設定したり、ページテーブルをたどったりする話が並びますが、芯にある問いは一つです。**安全な抽象化をどこまで Rust に任せ、どこから先を自分で責任を持つか** です。

OS 開発では、ハードウェアレジスタへの生ポインタ書き込みや、割り込みハンドラのような特殊呼び出し規約など、通常のアプリケーションでは触れない領域へ入ります。そこでは `unsafe` が必要になります。しかし `unsafe` は「何をしてもよい免罪符」ではありません。安全性の前提条件を自分で言語化し、その範囲だけを小さく閉じ込めるための印です。本書では毎回、どこが危険で、なぜその危険を局所化できるのかを追います。

この章の到達点

本書を読み終えるころには、少なくとも次の点を自分で説明できる状態を目指します。

- `#![no_std]` と `#![no_main]` を付ける理由
- ブートローダ、カーネル、CPU、メモリ管理の役割分担
- 例外と割り込みの違い

- x86_64 のページングが仮想アドレス変換をどう行うか
- ヒープを使う前に、なぜページを割り当てて写像する必要があるか

何を作るのか

本書で作るカーネルは、最終的に次のような流れで動きます。

1. ブートローダがカーネル ELF を読み込み、長いモードと基本的な実行環境を整える。
2. カーネルのエントリポイントが呼ばれ、最初の初期化コードが走る。
3. 文字出力機構を整え、画面やシリアルへ状態を出せるようにする。
4. 例外と割り込みのテーブルを初期化し、CPU の異常やデバイス入力へ反応できるようにする。
5. ページテーブルを操作し、仮想アドレス空間を自前で拡張する。
6. ヒープアロケータを有効化し、`Box` や `Vec` のような動的確保を使えるようにする。

ここまで辿り着くと、まだプロセス切り替えもファイルシステムもありませんが、「OS の核」と呼べる部分はかなり見えてきます。アプリケーションから見れば黒箱だった起動処理とメモリ管理が、自分で観察し、変更できる対象になります。

なぜ Rust なのか

OS 開発といえば C やアセンブリの印象が強いかもしれません。実際、ハードウェアとの境界では今でもそれらが重要です。それでも Rust を選ぶ利点は大きく、特に学習用途では有効です。

第一に、所有権と借用の制約が、雑な共有や寿命の曖昧さを早い段階で表面化してくれます。カーネルではグローバル状態や排他制御が多く登場しますが、そこで「本当に共有してよいのか」「どこまで可変なのか」を考えさせられるのは大きな助けです。

第二に、`unsafe` の境界が目印になります。C ではほぼ全文が危険領域ですが、Rust では「このブロックから先は前提を自分で保証する」という線を引けます。この線引きは教材として非常に扱いやすく、どこにハードウェア都合が染み出しているかを視覚化できます。

第三に、`x86_64` や `bootloader` をはじめとする `rust-osdev` 系のクレートが、素朴なラッパではなく、かなり良い抽象化を提供しています。もちろん魔法ではありませんが、学習の主題を「リンカ引数の泥臭さ」から「設計の意味」へ寄せやすくなります。

カーネルはどこにいるのか

アプリケーションプログラムは、すでに用意された OS の上で動きます。標準ライブラリ、ヒープ、スレッド、ファイル、時刻取得など、多くの機能は OS が提供します。カーネルはその OS の中核であり、CPU の特権命令、割り込み制御、ページテーブル、物理メモリ管理、デバイスアクセスのような責務を担います。

つまりカーネルを書くとは、普段は下に隠れている層を自分で引き受けることです。`println!` がなぜ画面へ出るのか、`Vec` がどこからメモリを受け取るのか、ページフォールトとは何に怒っているのか。これらは通常アプリケーションでは意識しなくて済みますが、カーネルでは自分で答えを持つ必要があります。

本書の範囲

本書では、次の範囲を本編に含めます。

- `no_std` のフリースタンディングバイナリ

- ブート可能な最小カーネル
- VGA テキストモードとシリアル出力
- `no_std` カーネル向けのテスト
- CPU 例外、ダブルフォールト、ハードウェア割り込み
- x86_64 ページングの理解と最小実装
- ヒープ領域の確保と単純なアロケータ

逆に、今回は次を扱いません。

- ユーザ空間とシステムコール
- スケジューラとマルチタスク
- ファイルシステム
- SMP と複数 CPU
- `async/await` による非同期タスク実行
- UEFI や BIOS の細部をゼロから実装する話

範囲を切る理由は、OS が広すぎるからです。広い分野を学ぶときは、最初に「芯の通った縦断面」を 1 本持つことが重要です。本書ではその縦断面を、起動からメモリ管理までに置きます。

> **注意**
> カーネル開発では、ツールチェインや周辺クレートの更新で手順が変わることがあります。この記事群の核は変わりませんが、補助 CLI や設定ファイルの細部は時期によって違います。本書では特定バージョンの写経よりも、「その設定が何のために必要か」を優先して説明します。

どのように読むか

OS 開発の説明は、章ごとの局所理解に落ちやすい分野です。VGA 出力の章ではメモリマップト I/O の話が中心になり、割り込みの章では IDT や PIC の話が中心になります。しかし、後半へ進むほど、各章の内容は相互に依存し始めます。たとえばページフォールトを正しく読むには、例外処理とページングの両方を理解している必要があります。

したがって本書では、各章を独立した小ネタとして読むのではなく、「制御権とメモリ空間を段階的に自分のものにしていく過程」として読むのが有効です。最初はブートローダに任せていたことが、途中からカーネル側の責務へ移ります。この責務の移動が見え始めると、各トピックが一本の流れとしてつながります。

`compilerbook` とのつながり

コンパイラ本と OS 本は別分野に見えて、実際には多くの接点があります。関数呼び出し規約、スタック、実行形式、静的リンク、型とメモリレイアウト、例外時の制御移動などです。`compilerbook` で「どういう命令列が出るか」を学んだあとに本書を読むと、「その命令列がどのアドレス空間で、どんな前提で動いているか」が見えてきます。

特に、次の対応を意識すると理解が深まります。

- `compilerbook` のコード生成と、本書の呼び出し規約や例外ハンドラの ABI
- `compilerbook` の実行イメージと、本書のページング・ELF ロード
- `compilerbook` のポインタと型、本書の仮想アドレス・物理アドレス・アロケータ

参考にしている公開 OSS

この文書の主参照は Philipp Oppermann 氏の `Writing an OS in Rust` と、その関連リポジトリです。加えて `rust-osdev` 組織のクレート群を念頭に置いています。これらがよい教材参照になる理由は、単に動くコードがあるからではありません。段階ごとに「いま何を抽象化したいのか」がはっきりしているからです。

小さな OS 教材は世の中に多くありますが、学習時に重要なのは、手順の列ではなく設計の理由が読めることです。`blog_os` はその点が優れており、Rust の安全性と x86_64 の泥臭さの折り合いを、比較的見通しよく追えます。

> **編集注記**
> 本書は `blog_os` を踏まえていますが、記事の全範囲を網羅することは目的にしていません。今回は `Boot` から `Paging` と `Heap` までを主線とし、そこから先の `Allocator Designs` や `Async/Await` は付録で見取り図だけを示します。

各章の見取り図

以下の順序で進みます。

1. まず、OS が使えない場所で Rust を起動するための最小条件を整えます。
2. 次に、画面やシリアルへ状態を出して、観察可能なカーネルを作ります。
3. そのうえで、テストの仕組みを入れ、変更を安全に積み上げられるようにします。
4. 例外と割り込みを処理して、CPU と外部イベントへ反応できるようにします。
5. ページテーブルを理解し、メモリ空間を自分で拡張できるようにします。
6. 最後にヒープを初期化し、動的確保を導入して、より普通の Rust コードへ近づけます。

この順序には理由があります。OS は最初から全機能を平行に作ると破綻しやすく、観察手段、異常時の逃げ道、メモリの土台という順で整えるのが安定だからです。次章では、その最初の一歩として、OS のない場所で Rust バイナリを立ち上げるところから始めます。

フリースタンディング Rust バイナリとブート

最初の仕事は、OS の上で動く普通の Rust バイナリから離れることです。カーネルを書くということは、標準ライブラリや C ランタイムが先に何かを整えてくれる前提を捨てることを意味します。ここでは、`no_std` と `no_main` から始めて、ブートローダ経由で最小カーネルを起動するところまでを整理します。

> **この章の到達点**
> この章では、次の 3 点を押さえます。
>
> - `std` と通常の `main` に何が含まれているか
> - カーネルが OS 非依存のバイナリである理由
> - ブートローダが先回りしてくれる役割

普通の Rust バイナリが暗黙に依存しているもの

通常の `cargo new` で作るバイナリは、`main` 関数だけ書けば動きます。しかしこれは、何もない場所から `main` が直接呼ばれているわけではありません。実際には、プログラム開始時にはランタイム初期化、スタックの準備、引数の受け渡し、終了処理など、さまざまな下支え

があります。さらに std はファイル I/O、スレッド、時刻、同期原語など OS に依存する機能を前提にしています。

カーネルは、その OS そのものを作る対象です。したがって、最初から std に寄りかかることはできません。ここで必要になるのが #![no_std] です。これによって、標準ライブラリを自動でリンクする前提を外し、core だけを基盤とする実行形へ切り替えます。

```
#![no_std]
#![no_main]

use core::panic::PanicInfo;

#[panic_handler]
fn panic(_info: &PanicInfo) -> ! {
    loop {}
}
```

この段階で重要なのは、「機能が減った」ことより「前提が減った」ことです。文字列出力やヒープ確保がなくなる一方で、どこまでが自分の責務かが明確になります。

no_main とエントリポイント

#![no_main] は、言語ランタイムに通常の main 呼び出しを生成させない指定です。アプリケーションではランタイムが _start から main へ橋渡しをしますが、カーネルではその橋渡し自体を自前で決めなければなりません。

この時点で _start のような生のエントリポイントを直接書く方法もありますし、後でブートローダの entry_point! マクロを使う形に寄せても構いません。教材としては両方を知っておく価値があります。前者は「CPU が最初にどこへ飛ぶか」を露わにし、後者は「最低限の儀式をラップする」設計を学べるからです。

パニック時に何をするか

panic! は通常、標準ライブラリやランタイムの助けを借りてメッセージ出力や巻き戻しを行います。しかし no_std カーネルではそのような支援がありません。したがって #[panic_handler] を自分で定義し、パニック時の挙動を決める必要があります。

最初は無限ループでも十分です。ただし実際には、後で VGA やシリアル出力を整えたら、ここで診断情報を出すように改善します。OS の開発では「失敗して止まる」ことそのものは普通で、重要なのは **止まる前に何を観測できるか** です。

ブートまでに必要な層

カーネルが動く前には、少なくとも次の層が並びます。

1. ハードウェアやファームウェアが最初の実行環境を与える。
2. ブートローダがカーネルイメージを読み込み、CPU モードやメモリ写像を整える。
3. カーネルがエントリポイントから制御を受け取り、自分の初期化を始める。

ここでブートローダは、単なるファイル読み込み係ではありません。x86_64 の長いモード移行、ページテーブルの初期設定、メモリマップの受け渡しなど、かなり面倒な下準備を肩代わりします。blog_os の第二版が教育的に優れているのは、この泥臭い部分を適度に隠し、主題を「カーネル設計」へ集中させている点です。

BIOS/UEFI をどこまで知るべきか

厳密には、PC の起動には BIOS や UEFI、ブートセクタ、EFI アプリケーション、長いモード移行など多くの要素があります。もちろん本格的に追う価値はありますが、最初の OS 教材で全部を同時に抱えると主題が散ります。

本書では、**ブートローダが何を保証してくれているか** を理解することを優先します。具体的には、カーネルコードが 64bit 環境で呼ばれ、必要なブート情報が引数で渡され、最低限のページテーブルが存在する。この契約を先に掴みます。ブートローダの内部をゼロから実装するのは、その契約が見えてからで十分です。

開発環境の考え方

実装に必要な道具は主に 3 つです。Rust のツールチェイン、QEMU のような仮想マシン、そしてブート可能イメージを作る補助ツールです。時期によって細かなコマンドは変わりますが、役割はほぼ固定です。

- Rust: カーネル本体を `no_std` でビルドする
- ブート支援ツール: カーネル ELF を起動可能なイメージへまとめる
- QEMU: 実機を汚さずに起動・停止・ログ観察を行う

```
rustup toolchain install nightly
rustup component add llvm-tools-preview
cargo build
qemu-system-x86_64 ...
```

コマンドそのものを暗記する必要はありません。重要なのは、どの段階で何をしているかです。`cargo build` はカーネル ELF を作るだけで、まだ「起動できるディスクイメージ」ではありません。そこへブートローダを組み合わせて、初めて VM が読む対象になります。

最小カーネルの形

ブートローダを使う場合、カーネル側の最小形はかなり簡潔です。典型的には、エントリポイントを登録し、引数でブート情報を受け取り、最後は `hlt` ループに落ちます。

```rust
#![no_std]
#![no_main]

use bootloader_api::{entry_point, BootInfo};
use core::panic::PanicInfo;

entry_point!(kernel_main);

fn kernel_main(_boot_info: &'static mut BootInfo) -> ! {
    loop {
        x86_64::instructions::hlt();
    }
}

#[panic_handler]
fn panic(_info: &PanicInfo) -> ! {
    loop {
        x86_64::instructions::hlt();
    }
}
```

このコードの本質は、「カーネル開始時の制御権を受け取り、その後は CPU 命令だけで待機できる」ことにあります。画面出力も割り込みもまだありませんが、すでに OS の上ではなく bare metal の世界へ入っています。

`hlt` ループの意味

最初の教材では `loop {}` と `hlt` ループが似て見えるかもしれません。しかし意味はかなり違います。空ループは CPU を常に回し続け、QEMU 上でも無駄に忙しくなります。`hlt` は次の割り込みまで CPU を待機状態へ入れる命令で、何もすることがないカーネルにとって自然な待ち方です。

この小さな差は重要です。OS では「何もしていない時間をどう扱うか」も設計対象になります。あとでタイマ割り込みやスケジューラへ進んだとき、`hlt` が自然に効いてきます。

ターゲット指定とリンク

カーネルはホスト OS 向けの通常ターゲットとは違う条件でビルドします。標準ライブラリ前提ではなく、実行時環境も異なるためです。初期の `blog_os` ではカスタムターゲット JSON やリンク設定の話が前面に出ますが、第二版ではブートローダや周辺クレートがこの負担をかなり吸収してくれます。

ここで学ぶべきなのは、設定ファイルの構文ではなく、**なぜ通常のデスクトップ向けターゲットでは不十分なのか** です。カーネルは OS サービスなしで起動しなければならず、実行ファイルの前提そのものが違います。したがってリンカの入口も、生成物の受け渡し方も、普通のアプリとは異なります。

> **注意**
> 公式記事やクレートの更新により、ターゲット JSON、`Cargo.toml`、補助 CLI の細部は時期によって変わります。本書では「ブートローダが起動可能イメージ化を担い、カーネルは `BootInfo` を受け取る」という設計を固定点として読みます。

最初に画面へ何か出す意味

カーネルを起動できるようになったら、すぐに画面出力を入れたくなります。これは単なる達成感のためではありません。OS 開発ではログ出力が生命線だからです。まだデバッガも整っておらず、例外ハンドラも未完成な段階では、「どこまで到達したか」を画面やシリアルで知るだけでも大きな前進です。

最小カーネルの時点では、画面へ直接メモリ書き込みする粗い実装でも十分です。むしろ最初はその粗さが大切で、文字が出るという事実を最短距離で得られます。そのうえで次章で安全な抽象化へ整理します。

観察可能性を最優先する

OS の学習でよくある失敗は、複雑な構成を先に整えすぎることです。たとえばメモリ管理や割り込みを先に厚く実装してしまうと、壊れたときにどこまで進んでいたかが分からなくなります。そこでこの段階では、以下を優先します。

- 起動できること
- 失敗時に停止できること
- 状態をどこかへ出せること

この 3 点が揃うだけで、以後のデバッグ効率は大きく変わります。カーネル開発では「正しさ」以前に「観測可能性」が重要です。

この段階での確認項目

最低限、次の確認ができれば次へ進めます。

- ブートローダ経由で QEMU がカーネルを起動する
- パニック時に無限再起動ではなく制御された停止になる
- カーネル本体のエントリポイントへ到達できている
- 何かしらの手段で「ここまで来た」が見える

これらは小さな確認ですが、後で割り込みやページングを入れるときの土台になります。どれかが曖昧なままだと、後半の不具合が「起動以前の問題」なのか「新機能の問題」なのか切り分けにくくなります。

次章への橋渡し

ここまでで、「OS のない場所で Rust コードを起動する」ことはできました。しかしまだ、成功も失敗も観測手段が乏しく、変更に対する自動チェックもありません。次章では VGA テキストモードとテスト基盤を整え、カーネルを **見えるもの**、**壊しても戻しやすいもの** にしていきます。

文字出力とテスト基盤

起動直後のカーネルで最初に欲しいのは、派手な機能ではなく観察手段です。画面へ文字を出し、必要ならシリアルへもログを流し、さらに変更を自動で検証する仕組みを持てれば、その後の開発速度は大きく上がります。この章では VGA テキストモードと `no_std` カーネル向けのテスト基盤を扱います。

> **この章の到達点**
> この章の核心は 2 つです。
>
> - ハードウェアに直接触る危険な操作を、小さな安全 API に閉じ込めること
> - カーネルでも回帰テストを回せる形を早めに作ること

VGA テキストモードはなぜ教材向きか

モダンな PC で最終的に使う表示系はもっと複雑ですが、学習初期の出力先として VGA テキストモードは非常に都合がよいです。理由は単純で、画面メモリが決まった物理アドレスへ対応しており、文字と属性を書き込むだけで結果が見えるからです。描画パイプラインやフォントレンダラを知らなくても、画面出力の最短経路を体験できます。

一方で、この単純さは危険でもあります。普通の参照や可変参照の感覚で生ポインタへアクセスすると、Rust の別名規則や最適化前提を破壊する可能性があります。したがって、最初から「とにかく書ければよい」ではなく、「unsafe をどうラップするか」を同時に考えるのが重要です。

`volatile` 書き込みの意味

メモリマップト I/O では、コンパイラ最適化が「この書き込みは後で読まれないから消してよい」と判断すると困ります。VGA バッファへの書き込みは、メモリの値そのものより副作用が本体だからです。そこで `volatile` アクセスを使い、書き込みが省略・並べ替えされにくい形にします。

```rust
#[repr(C)]
struct ScreenChar {
    ascii_character: u8,
    color_code: u8,
}

const VGA_BUFFER: *mut ScreenChar = 0xb8000 as *mut ScreenChar;
```

ここで大切なのは、危険なのが「アドレス 0xb8000」そのものではなく、そのアドレスに対して Rust の通常の参照規則を雑に適用することだという点です。したがって unsafe ブロックは必要最小限に留め、その外側では「行」「列」「色」「改行」といった意味のある API を提供します。

Writer 抽象化

典型的には `Writer` のような構造体を用意し、現在位置、色、バッファ参照を保持させます。公開 API には `write_byte`、`write_string`、`new_line` などを置き、外からは「画面へ文字列を書く」行為だけが見えるようにします。

この設計の利点は 2 つあります。第一に、危険なポインタ演算を一箇所に閉じ込められます。第二に、のちに出力先を VGA からシリアルやフレームバッファへ切り替えるときにも、呼び出し側の大半を保てます。OS 開発では、最初から抽象化のしすぎは禁物ですが、副作用の境界を隠す抽象化は早いほど効きます。

書式付き出力をどう実現するか

`println!` のような書式マクロは便利ですが、そのまま使えるわけではありません。そこで `core::fmt::Write` を実装し、自前の `print!` と `println!` マクロを用意するのが定番です。これにより、低レベルな書き込み機構と Rust の高水準な書式システムを接続できます。

```rust
impl core::fmt::Write for Writer {
    fn write_str(&mut self, s: &str) -> core::fmt::Result {
        self.write_string(s);
        Ok(())
    }
}
```

ここで面白いのは、OS 開発でも「全部アセンブリか生ポインタ」というわけではないことです。低レベルな出力経路の上に、高レベルな API を積み上げられるところは積み上げる。そのバランス感覚が Rust OS 開発の醍醐味です。

グローバル出力と排他制御

出力ライタは、多くの場合グローバルに共有したくなります。最小カーネルでは `static` なライタとロックを組み合わせる形がよく使われます。ここで初めて、OS における共有可変状態の扱いが顔を出します。

ロックを入れる理由は単にマルチスレッドだからではありません。割り込みハンドラや別経路のログ出力と出会ったとき、出力が中途半端に混ざるのを防ぐ必要があるからです。ただし OS では「ロック取得中に割り込みが入る」問題もあるため、後で割り込み無効化やロック順序の話と結び付きます。

シリアル出力も早めに持つ

画面出力だけでもよいように見えますが、実際にはシリアル出力があるとデバッグがかなり楽になります。特に画面まわりそのものが壊れているとき、VGA に依存しない経路でログを

取り出せる価値は大きいです。QEMU はシリアルを標準出力やファイルへ流しやすく、CI や自動テストとも相性がよいです。

教材として重要なのは、「出力先を複数持つと観測可能性が上がる」ことです。OS のバグは観測経路ごと壊すことがあるので、一つの手段に依存しないほうがよいです。

カーネルでのテストはなぜ難しいか

通常の Rust テストはホスト OS と `std` の上で動きます。しかしカーネルは `no_std` であり、終了コードの返し方も標準出力の扱いも違います。そのため、単に `cargo test` を打つだけでは不十分です。

ここで必要なのは、テストを **カーネル内で実行し、その結果を外へ報告する経路** です。`blog_os` ではカスタムテストフレームワークと QEMU の終了コード、シリアル出力を組み合わせ、各テスト関数を VM 内で実行する形を取ります。これは OS らしい発想で、テストランナ自体も実行環境の一部として組み込みます。

テストランナの最小像

テスト関数の列を受け取り、名前を表示し、1 件ずつ実行し、全部通れば所定の終了経路へ進む。概念だけ見れば単純です。

```
pub fn test_runner(tests: &[&dyn Fn()]) {
    serial_println!("running {} tests", tests.len());
    for test in tests {
        test();
    }
    exit_qemu(QemuExitCode::Success);
}
```

しかしこの単純さの裏には、いくつかの設計判断があります。テスト結果はどこへ出すのか。失敗時にどう停止するのか。QEMU の終了をどう表現するのか。パニック時に「テスト失敗」として報告するのか。こうした点を一つずつ自前で決める必要があります。

パニックとテスト失敗

ホスト環境の単体テストでは、失敗はランタイムが報告してくれます。カーネルでは、パニック時の振る舞いも自前です。そこでテスト用ビルドでは専用の panic handler を持ち、シリアルへ失敗メッセージを出して QEMU を失敗コードで終了させる構成にします。

この設計がよいのは、「パニックが起きた」という事実を **観測可能なテスト失敗** へ変換できる点です。以後、例外、ページング、ヒープ確保のような壊れやすい機能を追加しても、少なくとも前に通っていた振る舞いが壊れていないかを自動で見られます。

最初に書くべきテスト

本格的な機能を足す前でも、書けるテストはあります。たとえば次のようなものです。

- VGA 出力で文字列が折り返されるか
- `println!` が複数行を書いても壊れないか
- 期待した例外が起きたときにハンドラへ入るか
- パニック時にテスト失敗として終了するか

これらは地味ですが、土台の健全性を確認する役に立ちます。OS 開発では、派手なデモより「壊れていない」証拠のほうが価値が高い場面が多いです。

`unsafe` をどこへ閉じ込めるか

この章では `unsafe` が何度か出てきます。VGA バッファへのアクセス、ポート I/O、QEMU 終了のための特殊ポート書き込みなどです。重要なのは、`unsafe` が必要な箇所の多くが「ハードウェアに触る境界」であることです。

逆に言えば、その境界の内側を一度しっかり設計すれば、以後の大部分は安全な Rust のままで書けます。たとえば `println!` を使う側は生ポインタを意識しません。これは OS 開発でも、良い API が危険を局所化してくれるという証拠です。

画面出力で気を付ける落とし穴

VGA 出力は単純そうで、意外と落とし穴があります。

- 最適化で書き込みが消えないよう `volatile` を意識する
- 改行時のスクロール処理でコピー順序を壊さない
- 割り込み中のログ出力でロック再入やデッドロックを起こさない
- パニック中にさらに出力しようとして再帰的に壊れないようにする

こうした落とし穴は、後半の割り込み章でより重要になります。今のうちに「出力は単なる表示ではなく、カーネル全体から共有される副作用」と捉えておくと整理しやすくなります。

> **注意**
> 初期の教材では VGA が便利ですが、将来的に UEFI やより新しい描画方式へ進むと出力経路は変わります。それでも「危険な I/O を小さく包む」「観測手段を複数持つ」という設計原則は変わりません。

テストを先に整える価値

OS 開発では、動くデモを作る誘惑が強いです。しかしテスト基盤を後回しにすると、あとで例外やページテーブルを触り始めたときに急速に身動きが取れなくなります。理由は単純で、バグが出たときの切り分けコストが跳ね上がるからです。

たとえばページング章では、ヒープ初期化の前にメモリ写像を行います。ここで 1 ビット間違えるだけでページフォールトや再起動が起きます。そのとき、シリアルログとテストランナがあれば「どの初期化の直後で壊れたか」をかなり絞れます。前半でテストを入れるのは、後半の複雑さを先回りして抑えるためです。

この段階での到達基準

次へ進む前に、少なくとも次の状態にしておきたいです。

- 画面へ文字列を安定して出せる
- シリアルへログを流せる
- テスト用ビルドで QEMU を成功・失敗コード付きで終了できる
- パニック時の情報が観察可能である

ここまで整うと、以後のカーネルは「何か起きたら黙って止まる物体」ではなくなります。観察と回帰確認の足場ができたので、次章では CPU 例外とハードウェア割り込みへ進みます。

例外、ダブルフォールト、ハードウェア割り込み

ここから先は、CPU と外部イベントに反応する仕組みを整えます。これまではカーネルが一方的に処理を進めるだけでしたが、実際の OS では CPU 例外やタイマ、キーボード入力のような非同期イベントが常に割り込んできます。この章では、IDT、ダブルフォールト、PIC、タイマ、キーボードを足しながら、制御権の受け渡しを理解します。

この章の到達点

この章で押さえるべき軸は次の通りです。

- 例外と割り込みはどちらも CPU への制御移動だが、原因と意味が違うこと
- ハンドラは普通の関数ではなく、CPU との契約に従う必要があること
- ダブルフォールトやページフォールトのような異常系こそ、専用の備えが必要なこと

例外と割り込みの違い

例外は、現在実行中の命令や CPU の状態に起因して発生します。ゼロ除算、無効命令、ページフォールト、ブレークポイントなどが典型です。一方、割り込みは多くの場合、外部デバイスやタイマから非同期に通知されます。どちらも CPU が現在の実行を中断し、決められたハンドラへ飛ぶ点では似ていますが、「なぜ起きたか」と「再開の意味」は異なります。

この違いを意識しないと、デバッグ時に混乱します。たとえばブレークポイント例外は意図的に起こしてもよい観測手段ですが、ダブルフォールトは「例外処理そのものに失敗した」かなり危険な状態です。同じ「ハンドラへ飛ぶ」でも、意味の重さが違います。

IDT の役割

x86_64 では、例外や割り込みごとに対応するハンドラを Interrupt Descriptor Table に登録します。IDT は、何番のベクタが来たらどの関数へ制御を移すかという対応表です。OS の初期化でこの表を組み立ててロードすることで、CPU が異常や外部通知を受けたときの行き先を決めます。

Rust で扱うときに面白いのは、IDT エントリが「ただの関数ポインタ」ではないことです。特定の ABI、場合によってはエラーコード、割り込み無効化との関係など、CPU との契約が強くあります。x86_64 クレートが安全な型を提供してくれるのは助けになりますが、何を隠しているのかは理解しておく必要があります。

ブレークポイント例外から始める理由

最初の例外としてブレークポイントを扱うのは理にかなっています。ページフォールトや一般保護例外のようにシステム全体へ大きな影響を及ぼしにくく、意図的に起こしてハンドラが呼ばれるか確認できるからです。

この段階で大事なのは、IDT の初期化が正しく行われ、例外発生時に「再起動」ではなく「ハンドラへ到達」することを確認することです。OS 開発における大きな一歩は、異常を単なるクラッシュから **観察可能な制御移動** へ変えることです。

ダブルフォールトは何が怖いのか

ダブルフォールトは、ある例外を処理しようとしている最中に、さらに致命的な問題が起きたときに発生します。たとえば例外ハンドラが使うスタック自体が壊れていると、最初の例

外を処理することもできません。その結果、CPU はダブルフォールトを起こし、それすら処理できないと triple fault となってシステムが再起動します。

学習初期にこれが怖い理由は、症状が「何も言わずに再起動」に見えやすいことです。つまり観測可能性が一気に落ちます。したがってダブルフォールト対策は、異常系のための異常系ではなく、**デバッグ可能性を守る仕組み** として非常に重要です。

IST と専用スタック

ダブルフォールト対策の核心は、例外処理に失敗したときでも使える専用スタックを持つことです。x86_64 では TSS の Interrupt Stack Table を使い、特定の例外が来たときに別スタックへ切り替えることができます。これにより、「今使っていたスタックが壊れていたせいでハンドラへ入れない」という事態を避けやすくなります。

ここで見えてくるのは、OS におけるデータ構造の意味です。GDT や TSS は教科書的には少し古めかしい話に見えるかもしれませんが、実際には「CPU が制御移動するときにどの資源を使うか」を決める実践的な設定です。レジスタや命令だけでなく、例外時の土台も OS が整えます。

例外ハンドラで何を出すか

例外ハンドラでは、まず **静かに復旧しようとしない** ことが大切です。特にページフォールトやダブルフォールトの初期段階では、何が起きたかをシリアルや画面へ出し、必要なら停止したほうがよいです。曖昧なまま継続すると、後で別の場所で壊れて原因が見えなくなります。

ハンドラで役立つ情報は、例外種別、エラーコード、関連レジスタ、問題が起きたアドレス、スタックフレームなどです。これらを順に出せるようにしておくと、「IDT が壊れているのか」「ページテーブルが壊れているのか」「ただ意図通りにブレークポイントが入っただけなのか」を切り分けやすくなります。

ハードウェア割り込みの入口

例外の次は、タイマやキーボードのような外部イベントを扱います。x86 では伝統的に PIC が割り込みを CPU へ中継します。最近の実機では APIC 系が中心ですが、教材としては PIC が入り口として分かりやすく、QEMU でも扱いやすいです。

ここで重要なのは、「割り込みが来る前にハンドラとコントローラの両方を整える」ことです。ハンドラを登録しただけでは足りず、PIC 側でベクタ番号の割り当てやマスクを正しく設定し、最後に CPU の割り込み受付を有効にする必要があります。

タイマ割り込み

タイマ割り込みは、OS が時間を刻むための最小の拍子木です。今はスケジューラを持っていなくても、定期的にハンドラへ入れるだけで多くの確認ができます。たとえば画面の隅にドットを打つ、シリアルへ短い印を出す、カウンタを増やすといっただけでも、外部イベントの流れが見えてきます。

タイマ割り込みで注意すべきなのは、ハンドラの仕事を小さくすることです。重い処理やロック競合を持ち込むと、再入や応答遅延の問題がすぐ起きます。割り込みハンドラは「通知を受け取る場」であって、「重い処理を全部そこで行う場」ではありません。

キーボード割り込み

キーボード入力は、学習上とても良い題材です。割り込みを受け、ポート I/O からスキャンコードを読み、必要ならデコードして画面へ出す。これだけで「外部デバイスと会話している」感触が得られます。

一方で、ここには複数の低レイヤ要素が詰まっています。ポート I/O、ベクタ設定、割り込み応答、共有出力、入力デコードです。そのため教材としては、まずスキャンコードを読むところまでを確実にし、その後に文字変換やバッファ化へ進むほうがよいです。

End of Interrupt を忘れない

PIC を使う場合、ハンドラの最後で End of Interrupt を通知しないと、コントローラは「まだ処理中」だと思い込み、次の割り込みを正しく流さなくなります。これは非常に典型的な初学者バグで、最初の 1 回だけ動いたように見えたり、なぜか割り込みが止まったように見えたりします。

このような処理は地味ですが重要です。OS では「本体のロジック」よりも「相手との約束を守る儀式」が多く、その 1 つでも欠けると全体が静かにおかしくなります。

割り込みコンテキストでのロック

出力ライタやグローバル状態にロックを使っていると、割り込みハンドラ内で同じロックを取ってデッドロックする危険があります。たとえば通常コードがロックを保持したまま、途中でタイマ割り込みが入り、ハンドラでも同じ出力 API を呼ぶと詰みます。

そのため、割り込みコンテキストで共有資源へ触るときは次のいずれかを検討します。

- クリティカルセクション中は一時的に割り込みを無効化する
- 割り込みハンドラでは最小の情報だけ記録し、重い処理は後段へ回す
- 割り込み用に別の出力経路やロック戦略を持つ

この問題は OS らしい難所です。単に「Mutex があれば安全」ではなく、いつ誰がその Mutex を取りに来るかまで考える必要があります。

ページフォールトは後半の主役

この章ではまだページング実装へ入りませんが、ページフォールト例外は後半で主役になります。ページテーブルが未設定のアドレスへ触れた、書き込み禁止ページへ書いた、存在しない物理フレームを参照した。こうした不具合は、後半で最も頻繁に遭遇する失敗です。

したがって今のうちに、「例外ハンドラで問題のアドレスやエラー情報を出して止まる」という作法を身に付けておくのが重要です。ページング章では、この観測手段がそのまま切り分けの生命線になります。

> **注意**
> QEMU 上で「突然再起動する」症状は、ダブルフォールトや triple fault の可能性があります。まだログが出ていない段階では、原因を一つに決め打ちしないほうがよいです。まず IDT 初期化、専用スタック、パニック出力、シリアルログの順で観測点を増やします。

この章で得られる感覚

例外と割り込みが通るようになると、カーネルはようやく CPU と対話し始めます。今までは自分のコードが勝手に進むだけでしたが、ここからは「問題が起きたらどこへ飛ぶか」「外部イベントにどう応答するか」を自分で設計する段階です。

この感覚は、後のページングやメモリ保護でも重要です。OS とは、ただ命令を実行するプログラムではなく、**異常と非同期性を管理する主体** でもあります。

次章への橋渡し

ここまでで、制御移動の基盤は整いました。次はメモリ空間です。ブートローダが用意したページテーブルを理解し、自前でページを写像し、最後にヒープを有効化します。例外処理で整えた観測手段は、ページフォールトの切り分けでそのまま活躍します。

ページングとヒープ確保

カーネルが起動し、例外と割り込みも扱えるようになったら、次はメモリ空間を自分で制御できるようにします。この章では x86_64 のページングを理解し、最小限の写像操作を実装し、そのうえでヒープ領域を確保して動的メモリ割り当てを使えるようにします。

この章の到達点

この章の到達点は次の通りです。

- 仮想アドレスと物理アドレスの違いを説明できる
- ページテーブルをたどって写像を追加する意味が分かる
- ヒープ確保が「アロケータを置くだけ」ではなく、事前にメモリ写像を要することを理解する

なぜページングが必要なのか

ページングは、仮想アドレスと物理アドレスを分離し、保護と柔軟な配置を実現する仕組みです。もしプログラムが常に物理アドレスを直接使うなら、メモリ競合や断片化、保護違反の扱いが極めて難しくなります。OS はページングを使うことで、「この仮想領域をこの物理フレームへ対応付ける」「このページは実行禁止」「このページは読み取り専用」といった管理を行います。

初学者にとって重要なのは、ページングが単なる高速化機構ではなく **メモリに意味を与える管理層** だということです。ヒープ、カーネルコード、スタック、メモリマップト I/O は、同じ 64bit アドレスに見えても、背後では別の性質を持っています。その違いを OS が定義します。

x86_64 の 4 段ページテーブル

x86_64 では、通常 4 段のページテーブルを使って仮想アドレスを解釈します。P4, P3, P2, P1 と降りていき、最終的にページオフセットと組み合わせて物理アドレスを得ます。各段は 9bit のインデックスを持ち、1 ページは典型的に 4KiB です。

この構造は最初は少し機械的に見えますが、意図ははっきりしています。巨大なアドレス空間を疎に管理し、必要な表だけを作りながら階層的に対応付けるためです。教材としては、まず「どのビット列がどの表を選ぶか」を理解し、そのあとで実装 API と結びつけるのがよいです。

ブートローダが渡すもの

blog_os 系の流れでは、ブートローダがメモリマップや物理メモリへのオフセット情報をカーネルへ渡してくれます。これによりカーネルは、「今使っているページテーブルはどこにあるか」「どの物理領域が使用可能か」を知ることができます。

ここで重要なのは、ブートローダが **最初の答え** をくれているだけで、以後の管理まで代行してくれるわけではないことです。カーネルはその情報をもとに自前の Mapper や FrameAllocator を構築し、以後の写像や確保を自力で行います。

物理メモリをどう見えるようにするか

ページテーブルを操作したいのに、そのページテーブル自体は物理メモリ上にあります。ではカーネルはどうやってその中身へアクセスするのでしょうか。これがページング実装で最初にぶつかる壁です。

代表的な方法には、恒等写像、固定オフセット写像、全物理メモリ写像、再帰写像などがあります。教材として blog_os が採る「固定オフセットで物理メモリ全体を見えるようにする」方式は分かりやすく、後続の実装も素直です。物理アドレス p を仮想アドレス phys_offset + p で参照できるようにすると、ページテーブルの辿り方が整理しやすくなります。

Mapper の見方

x86_64 クレートの Mapper 系 API は、ページング操作をかなり安全に包んでくれます。Page、PhysFrame、PageTableFlags といった型が用意されており、「何をどこへ写像したいのか」を型付きで表現できます。

これは単なる便利ラッパではありません。OS 開発では、アドレス、サイズ、整列、権限ビットの取り違えが深刻なバグになります。型を経由することで、それらの一部をコンパイル時に押さえられるのは大きな利点です。とはいえ、最終的な前提の正しさはまだ人間が保証する必要があります。たとえば「この物理フレームは本当に未使用か」は型だけでは分かりません。

仮想アドレス変換を自分で辿る

最初に実装すると理解が深まるのが、与えられた仮想アドレスを現在のページテーブルで辿り、対応する物理アドレスを求める関数です。これは実用上も役立ちますが、それ以上に「4 段の表を通って翻訳する」という機械の動きを手で確かめる意味があります。

変換関数を書くと、次の点が自然に身に付きます。

- インデックスごとにページテーブルエントリを辿ること
- present ビットが立っていないときに変換が失敗すること
- 最後はフレーム先頭アドレスにページ内オフセットを足すこと

ページフォールトの切り分けでも、この感覚は非常に役立ちます。どの段で欠けたのかを想像できるだけで、問題の場所がかなり見えやすくなります。

新しい写像を作る

アドレス変換が理解できたら、次は未使用ページを物理フレームへ写像します。ここでは、仮想ページ、割り当てる物理フレーム、権限フラグ、フレームアロケータの 4 つが関わります。

典型的な流れは以下です。

1. 使いたい仮想ページ範囲を決める。

2. フレームアロケータから空き物理フレームを受け取る。
3. `PRESENT` や `WRITABLE` のフラグを付けて `map_to` する。
4. 必要なら TLB フラッシュを行う。

このとき、抽象的な「メモリを使いたい」が、実際には「未使用の仮想ページ」と「未使用の物理フレーム」を結び、権限を決める作業に分解されることが見えてきます。これが OS のメモリ管理の手触りです。

フレームアロケータ

ヒープや新規写像に必要な物理フレームは、どこから来るのでしょうか。答えはブートローダが渡したメモリマップです。そこには、どの物理領域が使用可能かが載っています。フレームアロケータはこの情報をもとに、未使用フレームを順に払い出します。

最初は単純な線形走査型で十分です。実用 OS では断片化や再利用、複数サイズ、NUMA など複雑な事情がありますが、学習の第一歩では「未使用フレームを二重に返さない」ことが最優先です。設計の芯は、ポインタの算術ではなく **所有権の一意性** にあります。

ページングと安全性

ページングまわりでは、`unsafe` の意味がさらに重くなります。誤った写像はただのロジックバグではなく、コード領域を上書きしたり、存在しないメモリへ飛んだり、予期せぬページフォールトを起こしたりします。したがって、危険箇所はできるだけ API の内部へ寄せ、外からは `Page` や `PhysFrame` のような型付き操作だけを見せるのがよいです。

ここで Rust が助けてくれるのは、「危険な前提をまとめて閉じ込める」ことです。もちろんページテーブルそのものの正しさはコンパイラが保証しませんが、アドレス種別の取り違えや整列のミスを減らすだけでもかなり助かります。

ヒープはなぜすぐ使えないのか

Rust で `Box` や `Vec` を使いたくなったら、アロケータを実装すればよいと思いがちです。しかし実際にはその前に、ヒープとして使う **仮想アドレス領域を物理メモリへ写像する** 必要があります。写像されていない領域へアロケータがポインタを返しても、最初のアクセスでページフォールトになるだけです。

この点は非常に重要です。ヒープ確保は論理的には 2 層あります。

- まず OS がヒープ用の仮想領域を準備し、ページを割り当てて使えるようにする
- その上でアロケータが、ヒープ内の空き領域を細かい要求へ分配する

多くの初学者は後者だけをイメージしますが、カーネル開発では前者こそ先に必要です。

`alloc` を使えるようにする

ヒープ用ページを写像したら、`GlobalAlloc` を実装したアロケータを登録し、`alloc` クレートを使えるようにします。最初のアロケータとしては bump allocator が分かりやすく、教材として十分です。単方向にポインタを進めていくだけなので、実装が短く、初期の検証にも向いています。

もちろん bump allocator には制約があります。個別解放が苦手で、長く使うには不向きです。しかしこの段階の主題は最強のアロケータを作ることではなく、「ヒープ初期化とアロケータ接続の全体像」を掴むことです。まずは `Box::new(41)` や `Vec::push` が動くところまで持っていくのがよいです。

どんなテストを書くべきか

この章のテストは、派手な統合テストより小さな確認が効きます。

- 仮想アドレス変換関数が既知の写像を正しく辿るか
- 新しく写像したページへ読み書きできるか
- ヒープ初期化後に `Box` や `Vec` が動くか
- 多数の割り当てで同じアドレスを誤って再利用しないか

特にヒープでは、「1 回動いた」だけでは不十分です。複数回の割り当て、異なるサイズ、再帰的なコレクション使用など、少し揺さぶりをかけるだけで粗いバグが見つかります。

ページフォールトをどう読むか

ページング章で最も頻繁に出会う失敗はページフォールトです。ここで重要なのは、ページフォールトを恐れるより、**読み解ける例外** にすることです。問題アドレス、エラーコード、アクセス種別、どの初期化直後に起きたかが分かれば、かなり絞れます。

典型的な原因は次の通りです。

- ヒープ用仮想領域をまだ写像していない
- ページ境界計算が 1 ページずれている
- 書き込み権限が足りない
- フレームアロケータが使用済み領域を再利用している
- `phys_offset` の前提が間違っている

これらは表面的な症状が似ていますが、観測点があれば整理できます。前章で例外処理を丁寧に整えた理由が、ここで効いてきます。

> **注意**
> ページングまわりの不具合は、見た目の症状が大きいわりに原因は小さいことが多いです。特にアドレス計算の off-by-one と権限フラグの不足は典型です。いきなり大きなヒープ初期化を疑う前に、1 ページだけの写像テストへ切り戻すと追いやすくなります。

ここまでで得られるもの

ヒープまで到達すると、カーネルはかなり「普通の Rust プログラム」に近づきます。もちろん `std` はなく、例外や割り込みやページテーブルを自分で抱えていますが、動的確保やコレクションを使えるだけで設計の自由度は大きく増します。

ただし、ここで忘れてはいけないのは、自由度の裏で OS 側の責務も増えていることです。`Vec` が動くのは、アロケータが正しく、ヒープが写像され、ページフォールト時の挙動が整っているからです。高水準な API の背後に、低水準な整備が何層もある。これがこの章の本質です。

次に進むなら

本編はここで一段落とします。この先は、より良いアロケータ、`async` タスク実行、キーボード入力の本格処理、ユーザ空間、システムコール、ファイルシステムといった方向へ伸ばせます。次章では、ここまでの実装を支えるデバッグ手順、演習、参考 OSS、次に読むべき題材をまとめます。

付録と次の一歩

本編では、起動、出力、テスト、例外、割り込み、ページング、ヒープまでを一気通貫で見ました。ここから先は、学習を続けるための補助線をまとめます。OS 開発は一見すると無数の枝道がありますが、実際には「観察可能性」「メモリ管理」「実行主体」の 3 本を軸に整理すると進みやすくなります。

推奨するコード構成

本書の内容を実装へ写すなら、最初から巨大な `lib.rs` へ書き続けるより、責務ごとに分けるほうが見通しがよくなります。たとえば次のような構成が自然です。

```
src/
  main.rs
  serial.rs
  vga_buffer.rs
  interrupts.rs
  gdt.rs
  memory.rs
  allocator.rs
  qemu.rs
tests/
  basic_boot.rs
  should_panic.rs
  stack_overflow.rs
```

この分割の意図は、単にファイルを綺麗に並べることではありません。`unsafe` やハードウェア境界をまとまったモジュールへ押し込み、テストの観測経路を明確にすることです。`memory.rs` はページテーブルとフレーム確保、`interrupts.rs` は IDT と PIC、`allocator.rs` はヒープと `GlobalAlloc` の境界を担います。

開発ループの基本形

OS 学習では、1 回の変更を小さく保つことが極端に重要です。おすすめのループは以下です。

1. 小さな変更を 1 つ入れる。
2. 画面またはシリアルへ到達確認を 1 か所足す。
3. 関連テストを回す。
4. 失敗したら、直前の最小差分へ戻って観測点を増やす。

このループは地味ですが、後半ほど効きます。ページングや割り込みでは、複数機能を同時に触ると原因が急に見えなくなるからです。

デバッグの進め方

OS のデバッグは、一般的なアプリケーションデバッグより「どの層で壊れたか」の見極めが重要です。おおまかには次の順で疑うと整理しやすいです。

1. そもそもブートローダからカーネルへ制御が渡っているか
2. 画面・シリアル出力はまだ生きているか
3. 例外ハンドラへ入れる状態か
4. ページテーブルや GDT/TSS の初期化が壊れていないか
5. 新しく触った共有状態やロックが再入で詰まっていないか

たとえば「何も表示されずに再起動する」なら、VGA 実装そのものを最初に疑うとは限りません。ダブルフォールトでログに届く前に落ちている可能性もあります。逆にシリアルには出るが画面だけ壊れるなら、出力サブシステム側へ絞れます。

QEMU とログの使い分け

QEMU は教材用途にとても向いています。起動が速く、クラッシュしても実機を巻き込まず、シリアルや終了コードも扱いやすいからです。学習段階では、次の 3 経路を持っておくとかなり楽になります。

- 画面出力: 到達点を直感的に確認する
- シリアル出力: テストや CI に流しやすい
- 終了コード: 自動テストの成功・失敗判定に使う

三者は役割が違います。画面は人間に優しいが自動化しにくく、終了コードは機械に優しいが情報量が少ない。両方を併用すると、短い開発ループを回しやすくなります。

よくあるつまずき

OS 初学者がよく遭遇する詰まりどころを挙げます。

- `panic_handler` を書いたのに、期待したメッセージが出る前に止まる
- VGA 出力は動くが、割り込み後に急に文字化けする
- 1 回だけ割り込みが入り、その後止まる
- ページング導入後、`Box` を作った瞬間にページフォールトになる
- ヒープは動くが、複数回確保すると不規則に壊れる

多くの場合、原因は複雑な理論ではなく、小さな契約違反です。EOI を忘れた、専用スタックが未設定、ページ範囲の終端が 1 つずれている、ロック中に割り込みが入った。そのような「儀式の漏れ」を疑う癖が役立ちます。

演習課題

本編の理解を固めるなら、次の順で課題を追加するとよいです。

1. シリアル出力へも `println!` 相当のマクロを揃える。
2. ブレークポイント例外のテストを独立した統合テストにする。
3. キーボード割り込みで受けた入力を単に表示するのではなく、リングバッファへ積む。
4. ヒープアロケータを bump から linked list 方式へ差し替える。
5. ページフォールトハンドラで問題アドレスとエラー種別を詳しく出す。

この順序には理由があります。前の課題ほど観測と基本 API の整備に近く、後ろほどメモリ管理と再入性の難しさが増えるからです。

この章の到達点

演習に入る前に、本編で最低限できていてほしいのは次の 4 点です。

- ブート可能なカーネルが QEMU 上で起動する
- 画面またはシリアルへ確実にログが出る
- 例外と最低 1 種類のハードウェア割り込みが通る
- ヒープ初期化後に `Box` と `Vec` が使える

章ごとの復習ポイント

ここまでの内容を短く復習すると、各章の核心は次のように整理できます。

- 導入: OS は「プログラムが動く場」を作る作業であり、Rust では unsafe の境界設計が中心課題になる。
- ブート: `std` と通常ランタイムを外し、ブートローダとの契約を通じて bare metal で実行を始める。
- 出力とテスト: 危険な I/O を小さな抽象化へ閉じ込め、観測可能性と回帰確認を早めに確保する。
- 割り込み: CPU とデバイスからの制御移動を自分で受け取り、異常や入力に応答できるようにする。
- ページングとヒープ: 仮想アドレス空間を自前で広げ、その上で高水準の動的確保を成立させる。

この復習を頭に置くと、新しい話題へ伸ばすときにも「今どの軸を触っているのか」が見えやすくなります。

用語小事典

本書で頻出した用語を短く整理します。

- `bare metal`: 既存 OS の上ではなく、ハードウェアに近い層で直接動くこと。
- `no_std`: 標準ライブラリへ依存しない Rust クレート。
- `panic_handler`: パニック発生時の振る舞いを定義する関数。
- `IDT`: 例外や割り込みベクタごとのハンドラを登録する表。
- `GDT`: セグメントや TSS 関連の設定に使う記述子表。
- `TSS`: タスク状態セグメント。x86_64 では IST 設定などに使う。
- `IST`: 例外時に切り替える専用スタックの仕組み。
- `PIC`: 伝統的なプログラマブル割り込みコントローラ。
- `Page`: 仮想アドレス空間上の固定サイズ領域。
- `Frame`: 物理メモリ側の固定サイズ領域。
- `Mapper`: ページとフレームの写像操作を表す抽象化。
- `FrameAllocator`: 未使用物理フレームを払い出す役割。
- `TLB`: 最近使ったアドレス変換を CPU が覚えておくキャッシュ。

ここから先に伸ばす方向

本編を終えたあと、学習の延長線は大きく 3 つあります。

第一はメモリ管理の強化です。より良いアロケータ、需要割り当て、コピーオンライト、ユーザ空間との分離などがこの方向に入ります。ページングを理解したあとに最も自然に伸ばせる道です。

第二は実行主体の導入です。タスク、スケジューラ、コンテキストスイッチ、システムコール、ユーザプロセスといった話がここに入ります。OS らしさが一気に増す一方で、例外とメモリ管理が前提になります。

第三はデバイスと I/O です。キーボードの本格入力、ストレージ、ファイルシステム、ネットワーク、PCI などです。ハードウェアとの境界設計が好きなら面白い方向です。

比較しながら学ぶとよい OSS

`blog_os` が一番の主参照ですが、理解を厚くするために並行して見る価値のある公開 OSS もあります。

- `rust-osdev/bootloader` ブートローダ側が何を肩代わりしているかを知るのに役立ちます。カーネルだけを見ていると、どこまで前提が与えられているか忘れやすいためです。
- `rust-osdev/x86_64` ページテーブル、IDT、レジスタ、命令ラッパがどのように型付けされているかを見ると、低レベル API の設計参考になります。
- `xv6` 言語は C ですが、プロセス、システムコール、ファイルシステムへ進みたくなったときの比較対象として非常に有益です。Rust 抽象化と対比しながら読むと勉強になります。
- `rCore` RISC-V 側の OS 教材として有名で、OS 教育の構成を比較するのに向いています。アーキテクチャは違っても、メモリ管理と実行主体の説明順は参考になります。

`compilerbook` との往復

この本を読んだあとに `compilerbook` へ戻ると、見え方が変わります。コンパイラが生成した関数呼び出し、スタック操作、グローバル変数配置、リンク結果が、実際にはどのような実行環境に置かれているかを想像しやすくなるからです。

逆に OS 側からコンパイラ本へ戻ると、「なぜ ABI や実行形式の理解が必要なのか」がより具体的に感じられます。この往復ができると、低レイヤ学習がばらばらの話ではなく、一つの連続した地図になります。

FAQ

Rust で OS を書くのは遠回りではないか

遠回りではありません。ハードウェア境界や unsafe の考え方を学ぶ必要はありますが、むしろ危険箇所を可視化しやすいぶん教材としての利点があります。C の素朴さには学習価値がありますが、最初の土台として Rust の型と所有権が助けになる場面は多いです。

最初からユーザ空間やファイルシステムまで進むべきか

おすすめしません。起動、観測、例外、ページング、ヒープの 5 本柱が揃っていない段階で範囲を広げると、原因不明の不具合が増えます。まずは一本の縦断面を作るほうがよいです。

実機で動かすべきか

学習初期は QEMU で十分です。実機は魅力的ですが、観測手段や復旧のしやすさを考えると、まずは仮想環境のほうが圧倒的に速く回せます。実機移行は、少なくともシリアルログと安定した起動手順を持ってからがよいです。

どうしてもダブルフォールトで落ちるときは

IDT, GDT, TSS, IST, パニック出力の順で観測点を確認します。専用スタックが効いていない、例外ハンドラ自体が壊れている、スタックオーバーフローが起きている、といった可能性が高いです。まずは最小構成へ戻し、意図的なブレークポイント例外が取れるかを確かめると整理しやすいです。

参考資料

本書を作る際に主に参照した公開資料は次の通りです。

- Philipp Oppermann, `Writing an OS in Rust` `https://os.phil-opp.com/`
- `phil-opp/blog_os` `https://github.com/phil-opp/blog_os`
- `rust-osdev/bootloader` `https://github.com/rust-osdev/bootloader`

- Rust OSDev organization `https://rust-osdev.com/`

> **編集注記**
> `blog_os` 本文と周辺クレートは継続的に更新されています。本書は 2026 年 3 月 10 日時点で公開されている第二版系の内容と `rust-osdev` 周辺の現行情報を踏まえて再構成しています。補助ツールやバージョン依存部分の細部は、実装時には公式ドキュメントを併読してください。

おわりに

低レイヤ学習で大切なのは、全部を知ることではなく、縦に一本通った理解を持つことです。コンパイラを作ると、プログラムを機械語へ変える視点が手に入ります。OS を作ると、その機械語がどういう環境で動くかの視点が手に入ります。この 2 つがつながると、普段のアプリケーション開発で見えている地面がかなり変わります。

本書が目指したのは、Rust と x86_64 の知識を広く散らすことではなく、「カーネルの最小骨格がどう立ち上がるか」を一本の流れで掴めるようにすることでした。ここまで読めたなら、次にどの分野へ進んでも、低レイヤの地図をかなり自力で広げていけるはずです。